backpropagation
mantığında şunu yaparız model yanlış tahmin yaptı nerede yanlış yaptı bunu anlayalım ve geri dönüp bunu düzeltmeye çalışalım

Backpropagation ve Chain Rule

Atıl Samancıoğlu

1 Backpropagation Nedir ve Neden Gereklidir?

Bir önceki dökümanda ANN mimarisini ve **forward propagation** adımını gördük. Forward propagation'ın yaptığı şey aslında çok basit:

- Girdileri içeri alır,
- Katmanlardan geçirir,
- En sonda bir tahmin üretir: $\hat{y}$.

Peki ya tahmin yanlışsa?

> *"Model yanlış tahmin yaptıysa, nerede hata yaptığını nasıl anlayıp nasıl düzelteceğiz?"*

İşte **backpropagation** bu sorunun cevabıdır. Çok kısa bir cümleyle:

> *Backpropagation, hatayı (loss'u) geriye doğru taşıyıp, hangi ağırlığın ne yönde güncellenmesi gerektiğini hesaplar.*

—

2 Loss (Hata) Fonksiyonu: "Ne Kadar Yanlışız?"

Modelin ne kadar iyi tahmin yaptığını anlayabilmek için bir ölçüye ihtiyacımız var. Bu ölçünün adı **loss function**.

Basit bir örnek olarak (şimdilik) şunu kullanalım:

$$L = (y - \hat{y})^2$$

Burada:

- y: gerçek çıktı
- $\hat{y}$: modelin tahmini

Bu formülü şöyle okuyabilirsin:

> *Tahmin ile gerçek arasındaki fark büyüdükçe, loss büyür.*

Eğitimin hedefi de zaten budur:

$$\min L$$

—

backpropagation mantığı şu ben
buradaki farkı alayım artık bir fark olduğunu biliyorum geri döneyim geri döneyim ağırlıkları
sonra bir daha aynısını baştan yapayım

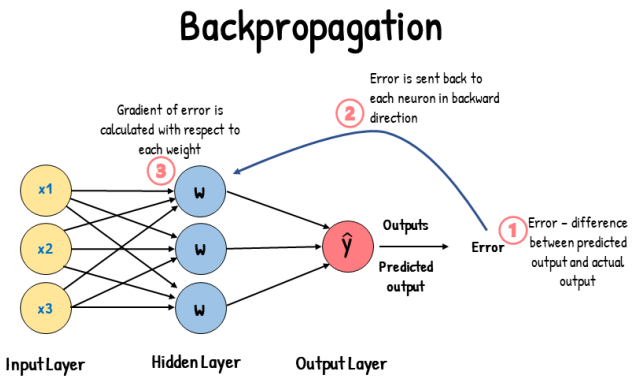


Figure 1: Backpropogation

3 Ağırlık Güncelleme: "Bir Sonraki Adımda Ne Yapmalıyız?"

Şimdi asıl meseleye geldik: Loss'u küçültmek için **ağırlıkları** güncellememiz gerekiyor.

En temel güncelleme formülü:

$$w_{\text{new}} = w_{\text{old}} - \eta \frac{\partial L}{\partial w}$$

Buradaki iki parça çok kritik:

- η (learning rate): adım büyüklüğü (ne kadar değişeceğiz?)
- $\frac{\partial L}{\partial w}$: yön (hangi tarafa gitmeliyiz?)

Bu formülün hissiyatı şudur:

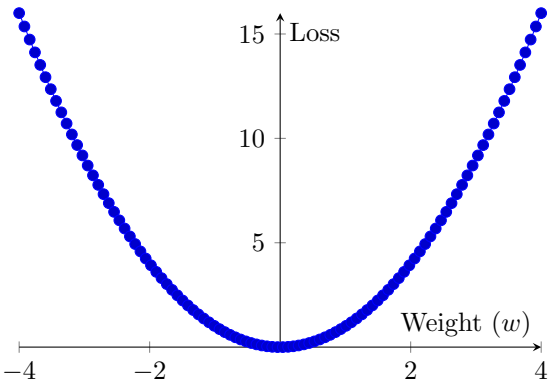
Loss'u artıran yönden uzaklaş, loss'u azaltan yöne yaklaş.

—

4 Neden Türev Alıyoruz? (Türev = Yön Bulma)

Türev kısmı ilk başta soyut gelebilir, ama aslında bir "yön bulma" aracıdır.

Loss'u tek bir ağırlığa göre çizdiğimizi hayal edelim (gerçekte binlerce ağırlık var ama mantık aynı):



Bu grafikte:

- Yüksek yerler: hata büyük
- En dip: hata minimum

Türev bize şunu söyler:

- Şu an bulunduğum noktada eğim pozitif mi negatif mi?
- Yani sağa gidersem mi aşağı inerim, sola gidersem mi?

Bu yüzden türev, "nereye gitmeliyim?" sorusunun matematiksel cevabıdır. Bu genel yaklaşımın adı da **gradient descent**.

—

5 Learning Rate Neden Önemli? (Adım Boyu Meselesi)

Learning rate (η) adım büyüklüğünü belirler. Aynı eğimde bile:

- Çok küçük η: güvenli ama yavaş ilerler
- Çok büyük η: hızlı gibi görünür ama hedefi kaçırabilir

Özet:

Learning rate, "ne kadar agresif öğreneceğim?" ayarıdır.

—

6 Backpropagation Mantığı (Forward Ayrı, Backward Ayrı)

Forward propagation: veriyi soldan sağa akıtır ve $\hat{y}$ üretir.
Backpropagation: loss'u baz alıp, ağırlıkları sağdan sola doğru günceller.

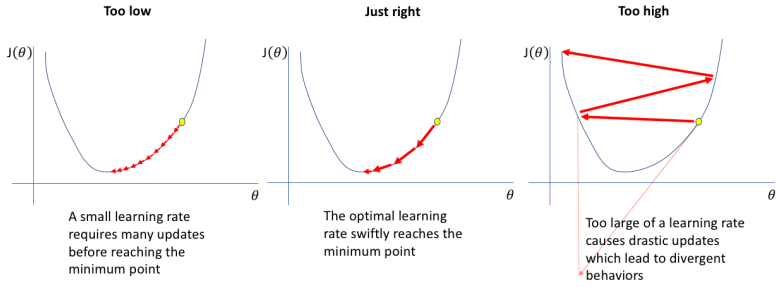
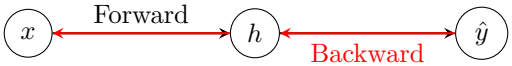


Figure 2: Learning Rate



L

Bir cümleyle:

> *Backpropagation, "bu hataya hangi ağırlık ne kadar sebep oldu?" sorusunu sayısallaştırır.*

—

7 Chain Rule: "Hata Bu Ağırlığa Nasıl Bağlanıyor?"

Chain rule kısmı genelde şu yüzden kafa karıştırır:

> *Loss'u ağırlığa göre türevleyeceğiz deniyor ama loss'un içinde doğrudan ağırlık yok gibi duruyor.*

İşte ana fikir şu:

7.1 Aslında Bir Zincir Var

Bir nöronda tipik akış:

$$w \to z \to \hat{y} \to L$$

Yani:

- w değişirse z değişir,
- z değişirse $\hat{y}$ değişir,
- $\hat{y}$ değişirse L değişir.

Dolayısıyla L, w'ye **dolaylı olarak** bağlıdır.
Bunu fonksiyon gibi yazarsak:

$$L = L(\hat{y}(z(w)))$$

7.2 Chain Rule Ne İşe Yarıyor?

Amacımız:

$$\frac{\partial L}{\partial w}$$

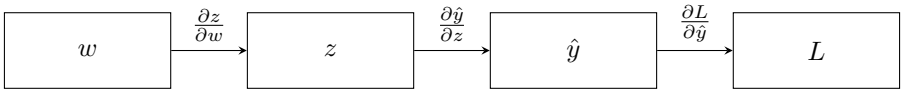
Ama L doğrudan w'den gelmediği için, aradaki halkaları kullanırız:

$$\frac{\partial L}{\partial w} = \frac{\partial L}{\partial \hat{y}} \cdot \frac{\partial \hat{y}}{\partial z} \cdot \frac{\partial z}{\partial w}$$

Bunu şöyle okuyabilirsin:

> *"Loss'un çıktı üzerindeki etkisi"* × *"çıktının z üzerindeki hassasiyeti"* × *"z'nin bu ağırlığa hassasiyeti"*

7.3 Görsel Olarak



Bu görsel şunu anlatıyor:

> *Loss'a ulaşmak için önce çıktıya, sonra z'ye, sonra ağırlığa geri bakmak zorundayız.*

—

8 Gizli Katman Ağırlıkları Neden Daha Zor? (Zincir Uzuyor)

Çıkış katmanındaki bir ağırlık genellikle daha "direkt" etkilidir. Ama gizli katmandaki bir ağırlık için zincir uzar çünkü o ağırlık:

- önce gizli nöronun çıktısını,
- o çıktı da sonraki katmanı,
- en son da loss'u etkiler.

Bu yüzden örnek bir gizli katman ağırlığı için türev şöyle uzar:

$$\frac{\partial L}{\partial w_1} = \frac{\partial L}{\partial \hat{y}} \cdot \frac{\partial \hat{y}}{\partial o_1} \cdot \frac{\partial o_1}{\partial z_1} \cdot \frac{\partial z_1}{\partial w_1}$$

Burada korkulacak bir şey yok; sadece şu var:

> *Hata, w_1'e gelene kadar kaç adım geçiyorsa, türevde o kadar çarpan olur.*

—

9 Backpropagation Ne Sağlar?

Backpropagation sayesinde:

- Ağırlıkları rastgele oynatmak yerine **hedefli** güncelleriz
- Her adımda loss'u genelde biraz daha düşürürüz
- Derin ağları pratik şekilde eğitebiliriz

—

10 Kısa Özet

- Loss: "Ne kadar yanlışız?"
- Türev: "Hangi yöne gitmeliyiz?"
- Learning rate: "Ne kadar büyük adım atalım?"
- Chain rule: "Hata bu ağırlığa hangi yoldan geliyor?"

Bir sonraki dökümanda (öneri):

- Loss fonksiyonlarını (MSE vs Cross-Entropy)
- Aktivasyon fonksiyonlarını (Sigmoid, ReLU, Softmax)
- Optimizasyon algoritmalarını (GD, SGD, Adam)

ayrı ayrı, bol örnekle inceleyeceğiz.